

Data and Artificial Intelligence

Cyber Shujaa Program

Week 8 Assignment

Classification Models

Student Name: Violet Joy

Student ID: cs-da01-25025

Table of Content

Data and Artificial Intelligence	1
Cyber Shujaa Program.....	1
Week 8 Assignment Classification Models.....	1
Introduction	3
Tasks Completed	4
2.1: Load the dataset	4
2.2: Explore the Dataset.....	5
2.3: Prepare the Data	10
2.4: Build the Models	12
2.5: Evaluate the Models	14
2.6: Compare the Models	18
2.7: Share Your Work.....	23
Conclusion.....	25

Introduction

In this project, I explore the application of supervised machine learning techniques to a multi-class classification problem using the Wine dataset from the scikit-learn library. The dataset contains chemical analysis results of wines derived from three different cultivars, with the objective of classifying each wine sample into its respective class based on a set of numerical features.

To achieve this, six popular classification models are implemented and evaluated: Logistic Regression, Decision Tree, Random Forest, k-Nearest Neighbors (KNN), Naive Bayes, and Support Vector Machine (SVM). The process involves exploratory data analysis (EDA) to uncover feature distributions and relationships, followed by preprocessing steps like data cleaning and splitting into training and test sets.

Each model is trained under the same conditions and assessed using standard evaluation metrics including accuracy, classification reports (precision, recall, F1-score) and confusion matrices. The ultimate goal is to compare the performance of these models and determine which algorithm is best suited for this dataset, providing reasoning for the results based on model behavior and data characteristics.

This assignment not only reinforces the fundamental steps of building machine learning models but also emphasizes the importance of evaluating and comparing model effectiveness in a consistent and interpretable manner.

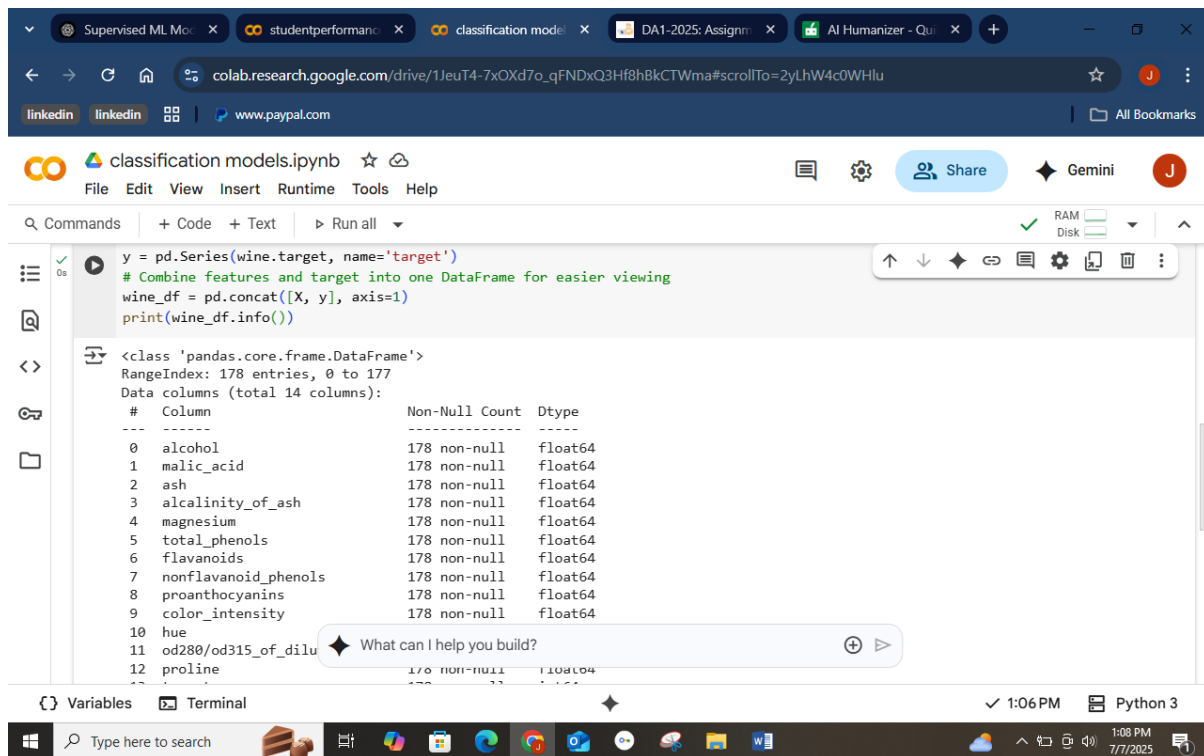
Tasks Completed

2.1: Load the dataset

The first step in this project is to load the Wine dataset, which is available directly from the `sklearn.datasets` module. This dataset contains 178 observations of wine samples classified into three distinct categories based on 13 numerical features related to chemical composition.

Using the `load_wine()` function, I import the dataset into our environment. The data is then converted into a structured pandas DataFrame for easier analysis and visualization. The feature matrix (X) contains the input variables, while the target vector (y) contains the class labels corresponding to the type of wine.

This step sets the foundation for the entire machine learning pipeline by providing structured and labeled data for analysis and model training



```

y = pd.Series(wine.target, name='target')
# Combine features and target into one DataFrame for easier viewing
wine_df = pd.concat([X, y], axis=1)
print(wine_df.info())

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   alcohol                             178 non-null    float64
 1   malic_acid                          178 non-null    float64
 2   ash                                 178 non-null    float64
 3   alcalinity_of_ash                   178 non-null    float64
 4   magnesium                           178 non-null    float64
 5   total_phenols                       178 non-null    float64
 6   flavanoids                          178 non-null    float64
 7   nonflavanoid_phenols                178 non-null    float64
 8   proanthocyanins                     178 non-null    float64
 9   color_intensity                     178 non-null    float64
10   hue                                 178 non-null    float64
11   od280/od315_of_diluted_sou         178 non-null    float64
12   proline                             178 non-null    float64

```

Fig 2.1.1: Output of loading the dataset and converting data type

2.2: Explore the Dataset

In this phase, I perform Exploratory Data Analysis (EDA) to gain a deeper understanding of the Wine dataset. EDA helps reveal underlying patterns, relationships between features, and the distribution of target classes. This step is crucial for identifying potential issues, such as imbalance in class distribution or highly correlated features.

I begin by examining the structure of the dataset using functions like `head()`, `.info()`, and `describes()` to understand the types of features and their statistical summaries. I then check the distribution of the target classes to ensure there is no significant class imbalance that could affect model performance.

Visualizations such as:

Histograms: Show the distribution of each numerical feature across all samples.

Box Plots: Compare how features vary across the different wine classes.

Pair Plot: Visualizes pairwise relationships for a subset of features, colored by class.

Correlation Heat map: Highlights the strength of relationships between features to help detect multicollinearity.

Overall, this step provides the foundation for informed data preprocessing and model selection by highlighting the characteristics and structure of the dataset.

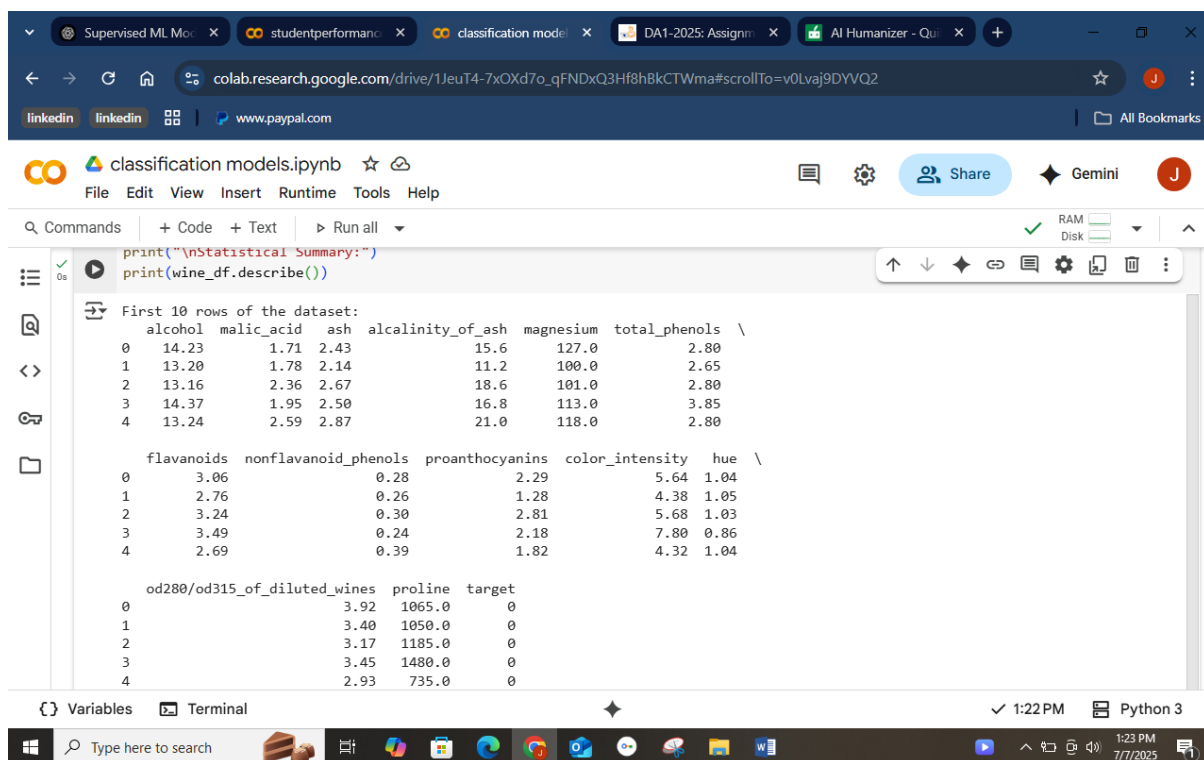


Fig 2.2.1: Output of the first ten rows of the dataset

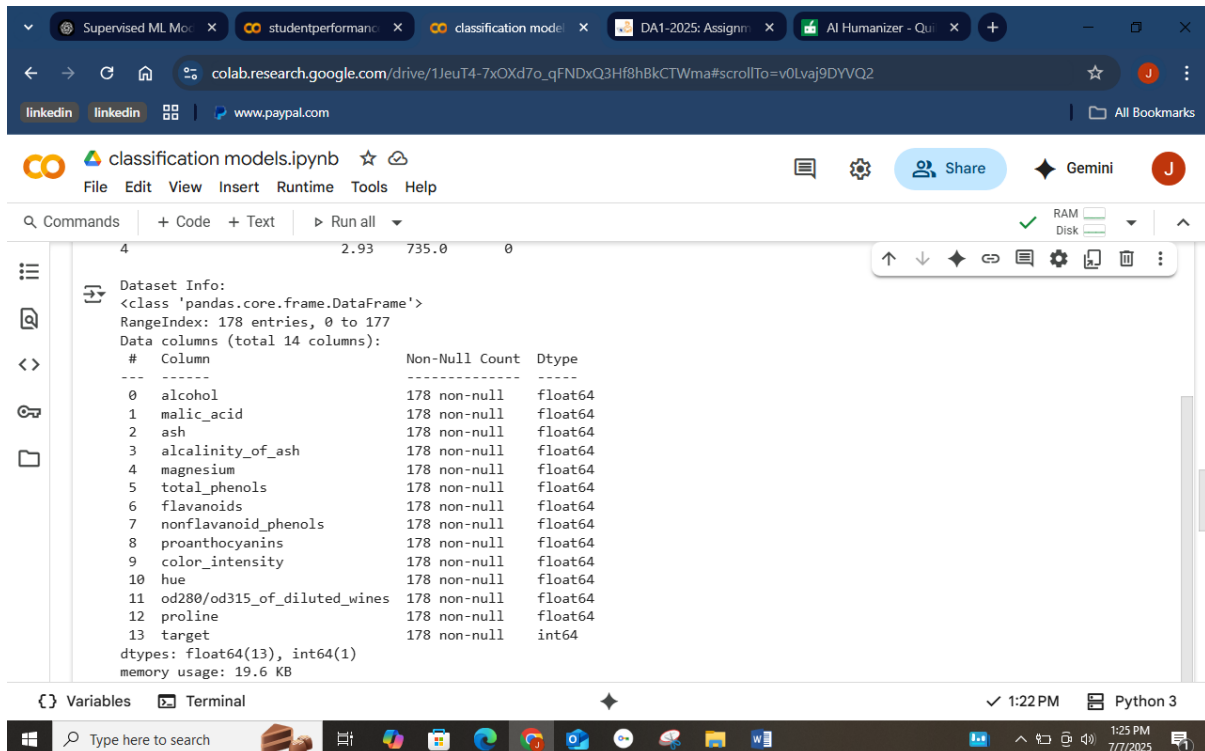


Fig 2.2.2: Output of the dataset info

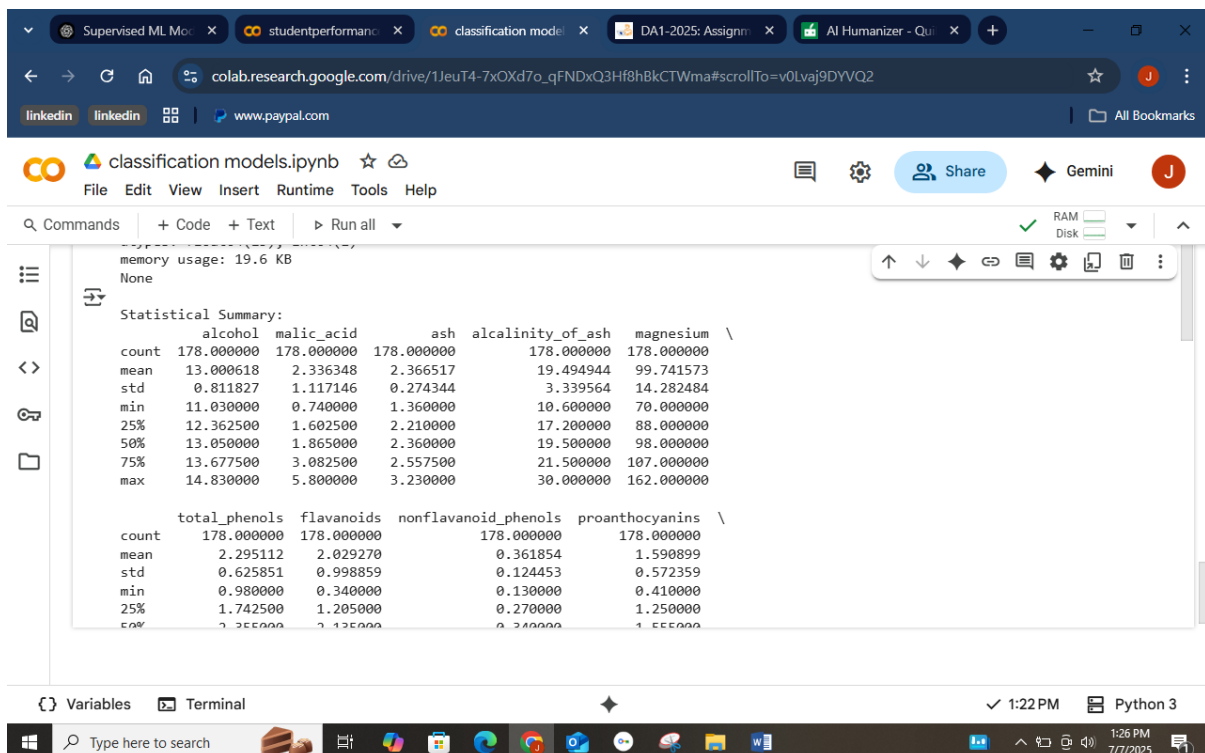


Fig 2.2.3: Output of summary statistics

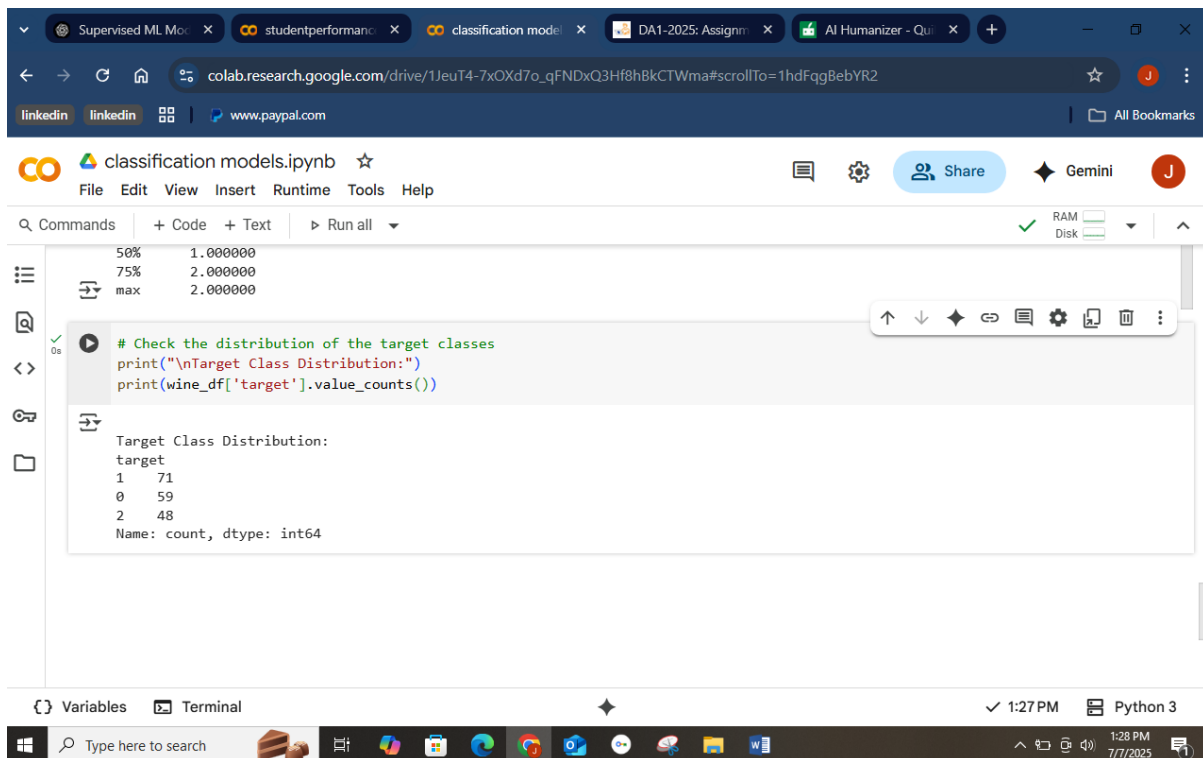


Fig 2.2.4: Output of target class distribution

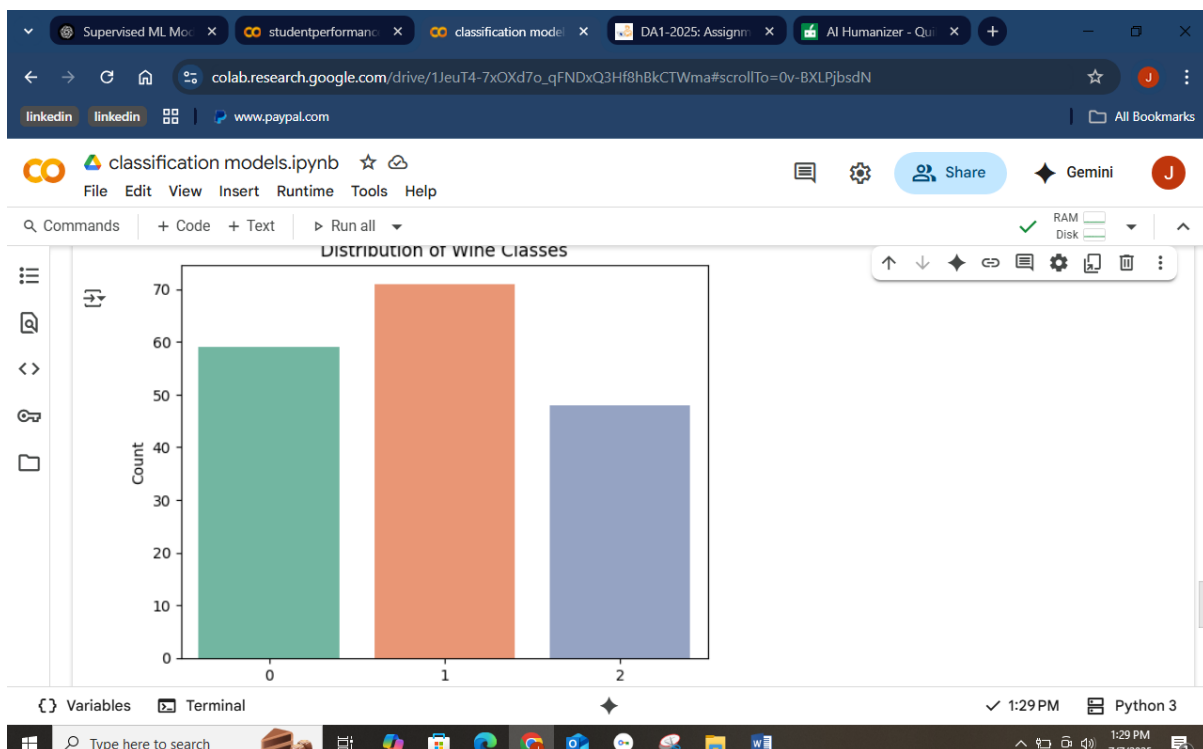


Fig 2.2.5: Output of bar plot of distribution class against count

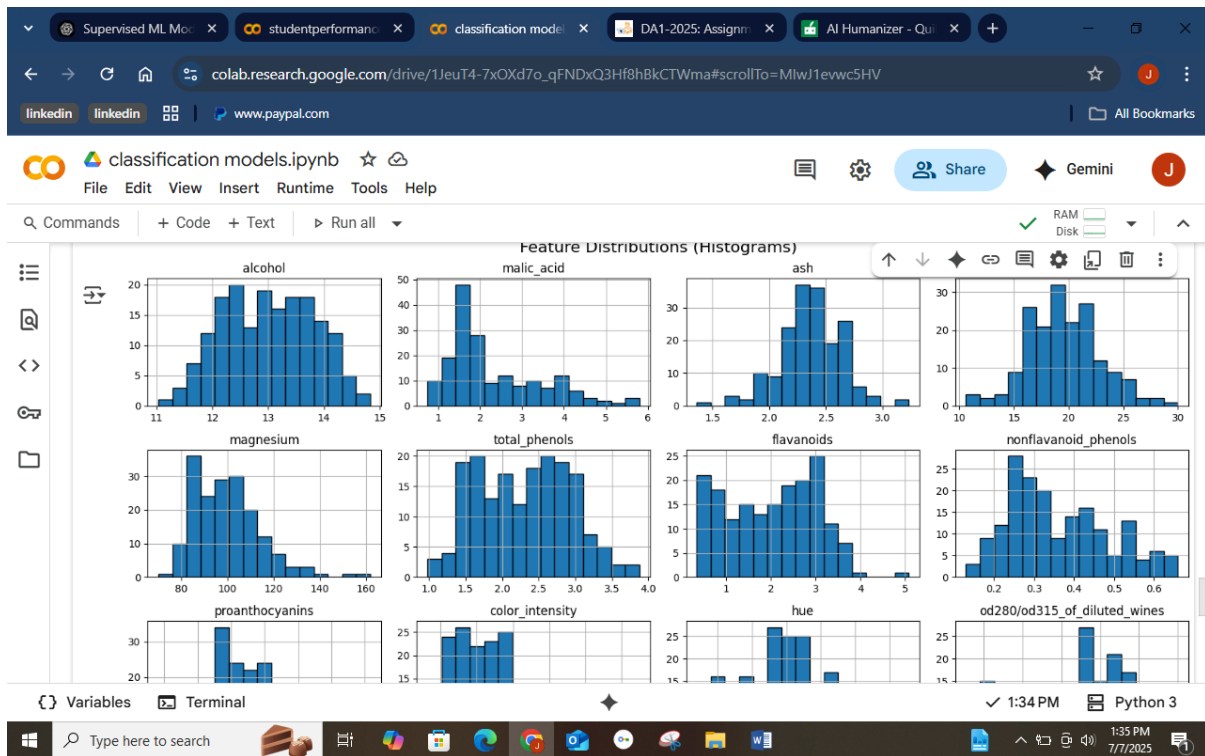


Fig 2.2.6: Output of feature distribution (histogram)

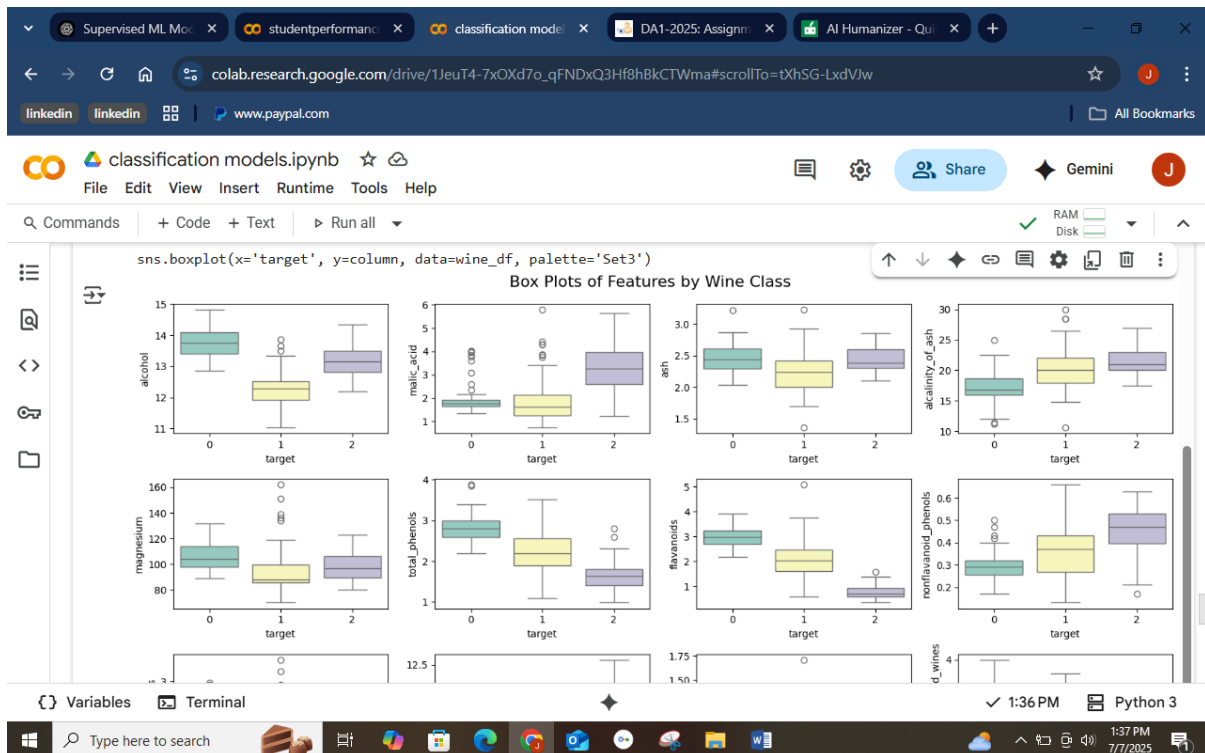


Fig 2.2.7: Output of boxplot of features by wine class

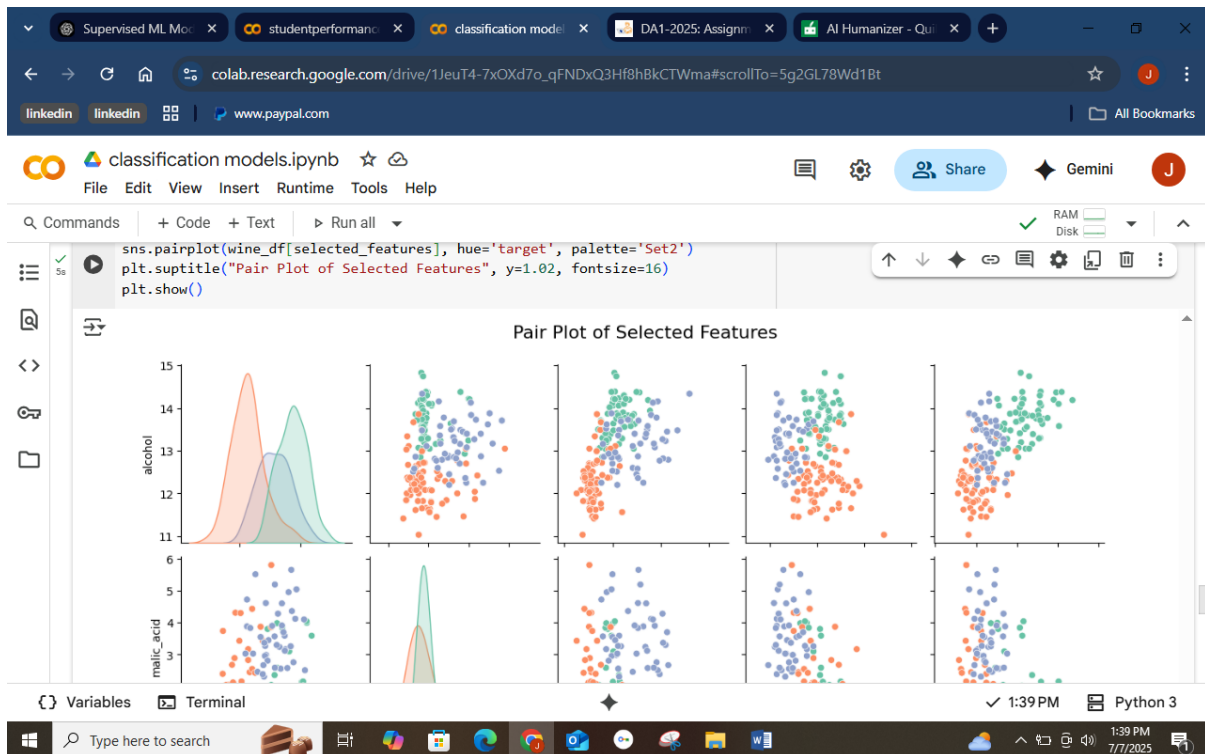


Fig 2.2.8: Output of pair plot of selected features



Fig 2.2.9: Output feature correlation heat map

2.3: Prepare the Data

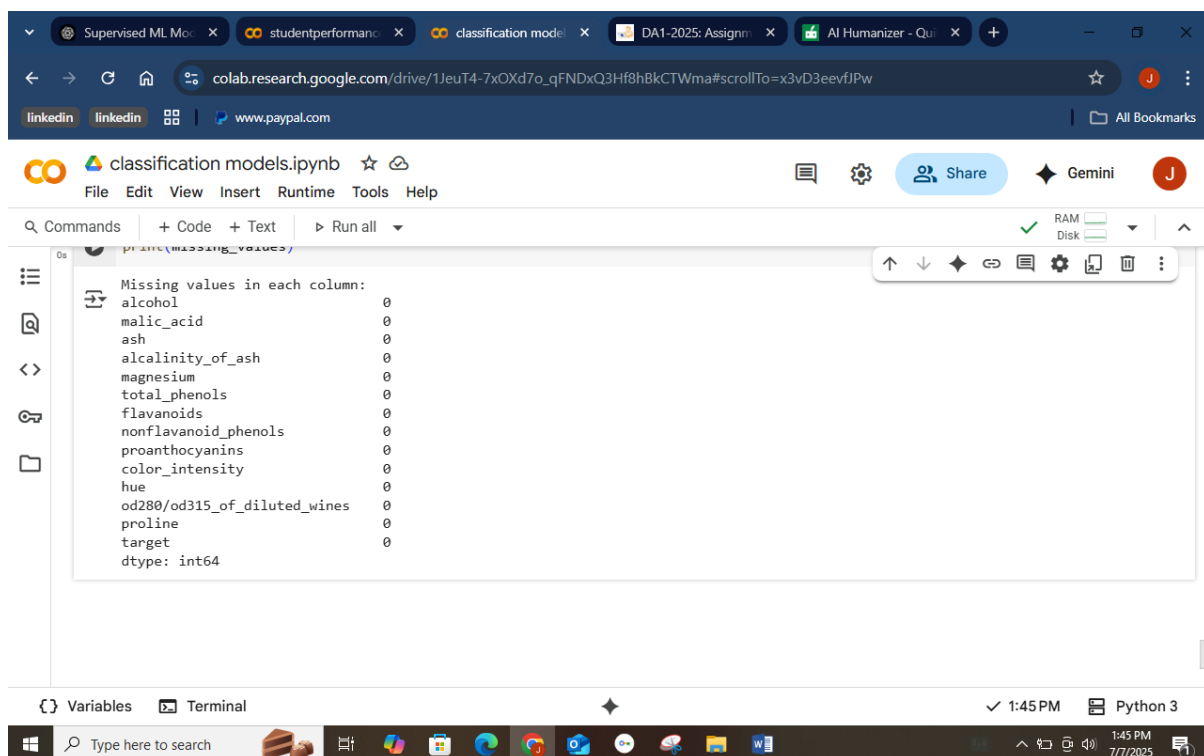
Before training our models, I need to prepare the dataset to ensure it's clean and suitable for machine learning algorithms. The preparation process involves two key steps: handling missing values (if any) and splitting the dataset into training and test sets.

First, I check for missing values in the dataset using methods such as `.isnull().sum()`. Fortunately, the Wine dataset does not contain any missing values, so no imputation or removal is needed.

Next, I separate the dataset into input features (X) and target labels (y). Using `train_test_split` from `sklearn.model_selection`, I divide the data into training and test sets—typically using an 80/20 or 70/30 split. This ensures that the models can be trained on one portion of the data and evaluated on unseen data, helping to assess their generalization performance.

I also apply feature scaling (standardization) using `StandardScaler`, which ensures that all features contribute equally to the model's learning process—especially important for distance-based models like KNN and SVM.

By completing this step, the dataset is cleaned, structured, and ready to be used in building and training the classification models.



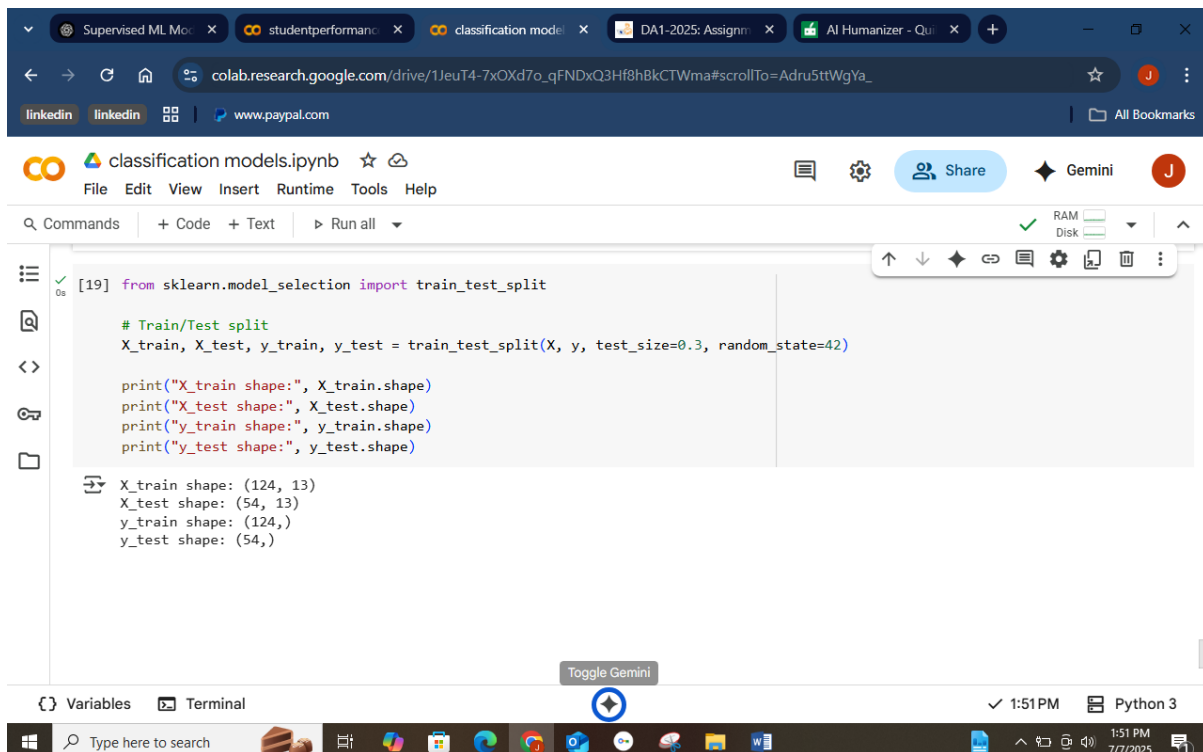
```

print(missing_values)

Missing values in each column:
alcohol                0
malic_acid             0
ash                   0
alcalinity_of_ash      0
magnesium              0
total_phenols          0
flavanoids             0
nonflavanoid_phenols   0
proanthocyanins        0
color_intensity        0
hue                   0
od280/od315_of_diluted_wines  0
proline                0
target                0
dtype: int64

```

Fig 2.3.1: Output of no missing values



The screenshot shows a Google Colab notebook titled "classification models.ipynb". The code in cell [19] imports `train_test_split` from `sklearn.model_selection` and splits the data into training and testing sets. The output shows the shapes of the resulting arrays.

```
[19] from sklearn.model_selection import train_test_split

# Train/Test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

X_train shape: (124, 13)
X_test shape: (54, 13)
y_train shape: (124,)
y_test shape: (54,)

Fig 2.3.2: Output of splitting and scaling the dataset

2.4: Build the Models

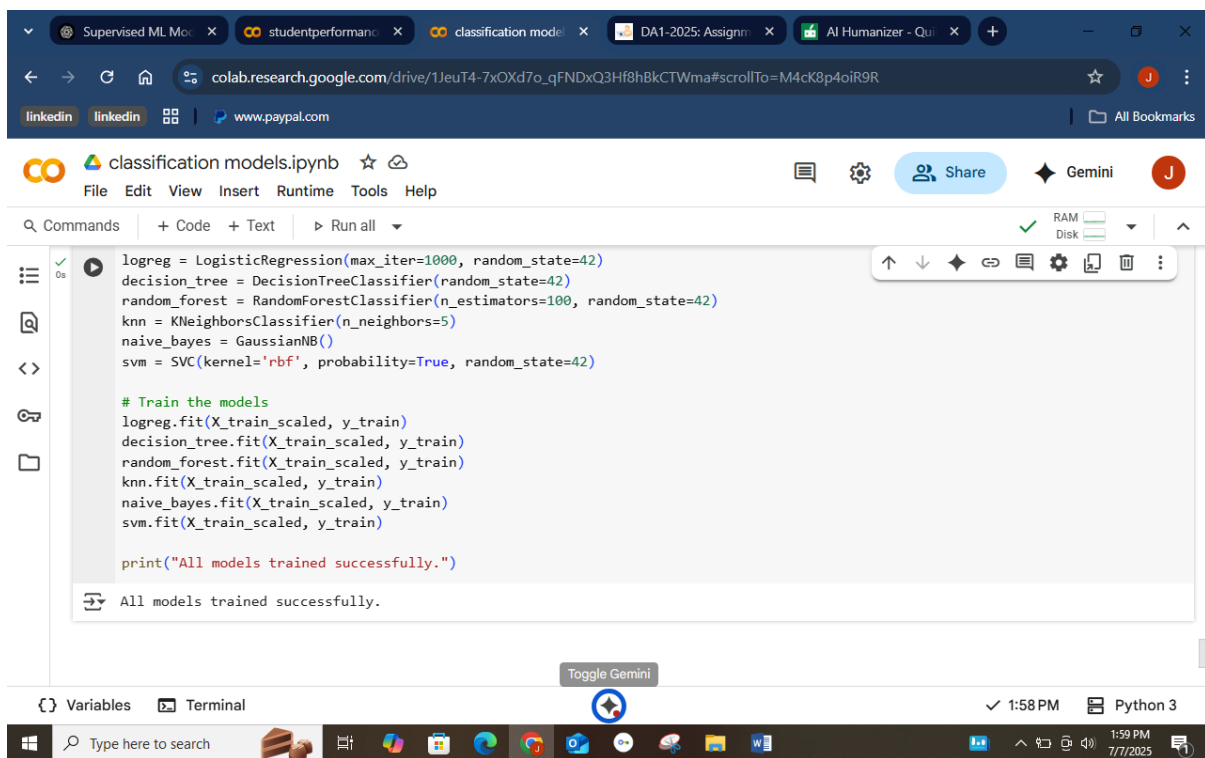
In this step, I implement and train six different supervised machine learning classification models using the prepared training dataset. Each model uses a different algorithmic approach to learn patterns from the data, which allows for comparison and deeper understanding of their performance.

The models built include:

- **Logistic Regression:** A linear model commonly used for binary and multi-class classification problems. It estimates the probability of a class label using the logistic function.
- **Decision Tree:** A tree-based model that splits the data into branches based on feature values, creating a series of decision rules to classify data points.
- **Random Forest:** An ensemble method that builds multiple decision trees and combines their predictions for better accuracy and reduced overfitting.
- **k-Nearest Neighbors (KNN):** A non-parametric model that classifies a data point based on the majority class among its k closest neighbors in the training data.
- **Naive Bayes:** A probabilistic classifier based on Bayes' Theorem, which assumes that features are conditionally independent given the class label.
- **Support Vector Machine (SVM):** A powerful model that finds the optimal hyperplane to separate classes by maximizing the margin between them.

Each model is initialized using scikit-learn's built-in classifiers and trained using the `.fit()` method on the training data. This step allows the models to learn the underlying patterns and relationships between the features and their corresponding class labels.

By building multiple models, I can later compare their strengths and weaknesses based on performance metrics, helping to determine the most suitable model for this classification task.



The screenshot shows a Google Colab notebook titled 'classification models.ipynb'. The code in the notebook initializes six classifiers: LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, KNeighborsClassifier, GaussianNB, and SVC. Each classifier is then trained using the `.fit()` method on `X_train_scaled` and `y_train`. A print statement at the end confirms that all models were trained successfully.

```
logreg = LogisticRegression(max_iter=1000, random_state=42)
decision_tree = DecisionTreeClassifier(random_state=42)
random_forest = RandomForestClassifier(n_estimators=100, random_state=42)
knn = KNeighborsClassifier(n_neighbors=5)
naive_bayes = GaussianNB()
svm = SVC(kernel='rbf', probability=True, random_state=42)

# Train the models
logreg.fit(X_train_scaled, y_train)
decision_tree.fit(X_train_scaled, y_train)
random_forest.fit(X_train_scaled, y_train)
knn.fit(X_train_scaled, y_train)
naive_bayes.fit(X_train_scaled, y_train)
svm.fit(X_train_scaled, y_train)

print("All models trained successfully.")
```

The output of the notebook shows the message: "All models trained successfully."

Fig 2.4.1: Output of all trained model successful

2.5: Evaluate the Models

Once the models are trained, I evaluate their performance on the test set using standard classification metrics. Evaluation helps us understand how well each model generalizes to unseen data and provides insights into their strengths and weaknesses.

The following metrics are used:

- **Accuracy Score:** Measures the overall proportion of correctly predicted instances. It gives a quick overview of the model's effectiveness but may not be sufficient for imbalanced datasets.
- **Classification Report:** Includes precision, recall, and F1-score for each class:
 - **Precision** indicates the percentage of true positive predictions out of all positive predictions made.
 - **Recall** measures the model's ability to correctly identify all actual positives.
 - **F1-Score** provides a balance between precision and recall, especially useful when class distribution is uneven.
- **Confusion Matrix:** A table that visualizes the performance of the model by showing the actual vs. predicted classifications. It helps identify specific classes the model may be confusing with others.

In addition to the numeric output, **visual plots of the confusion matrices** are generated using Seaborn's heatmap() for clearer interpretation of misclassifications. These visualizations make it easier to spot performance trends and areas for improvement.

By applying these metrics consistently across all six models, I am able to perform a fair and detailed comparison of their performance.

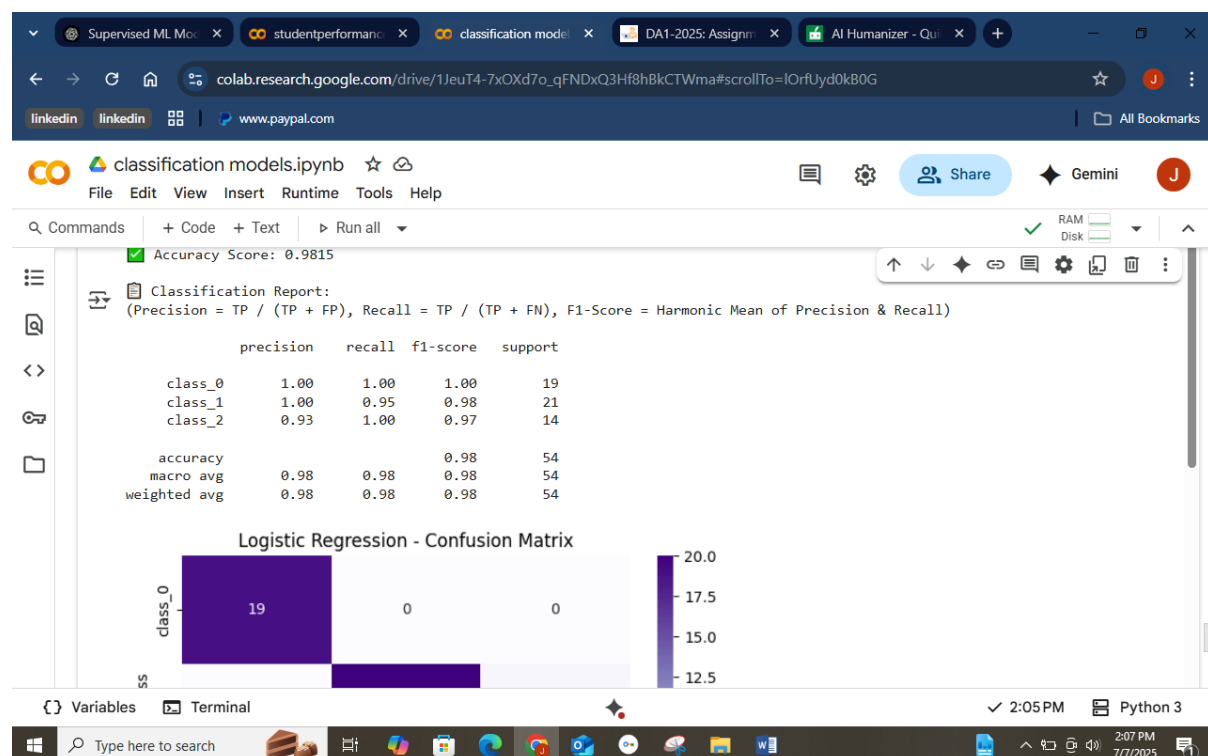


Fig 2.5.1: Output of logistic regression

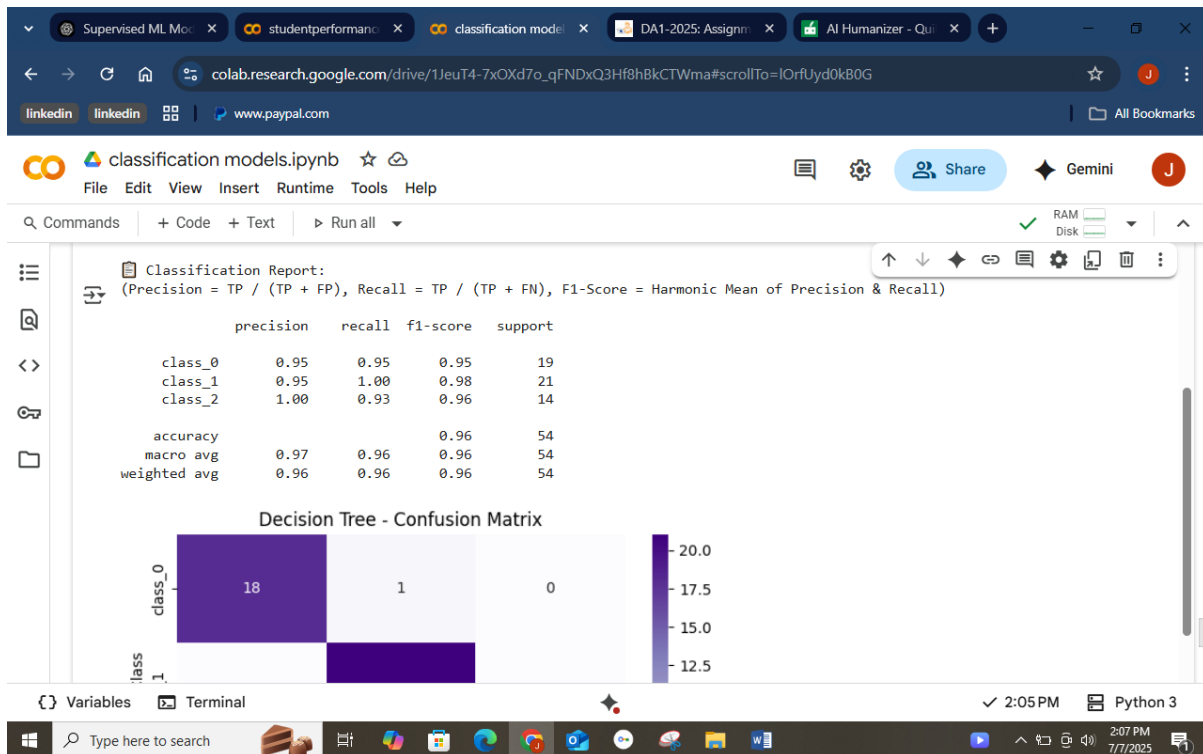


Fig 2.5.2: Output of decision tree

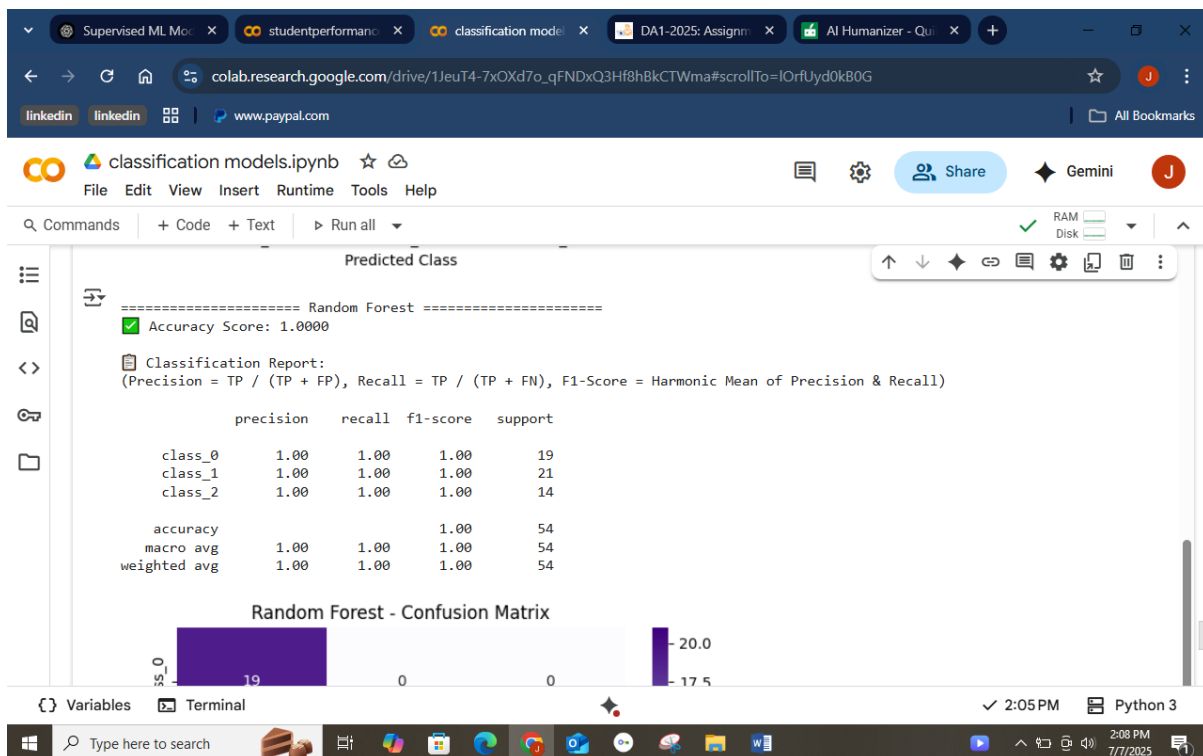


Fig 2.5.3: Output of random forest

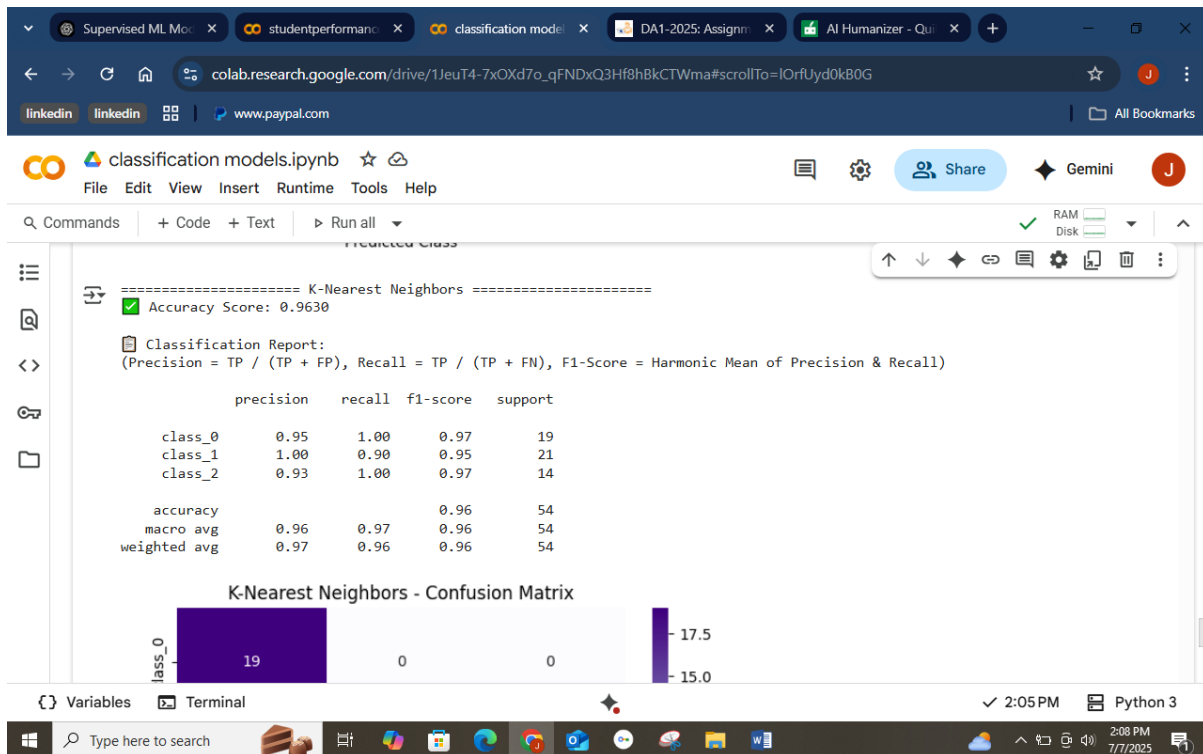


Fig 2.5.4: Output of K-Nearest Neighbours

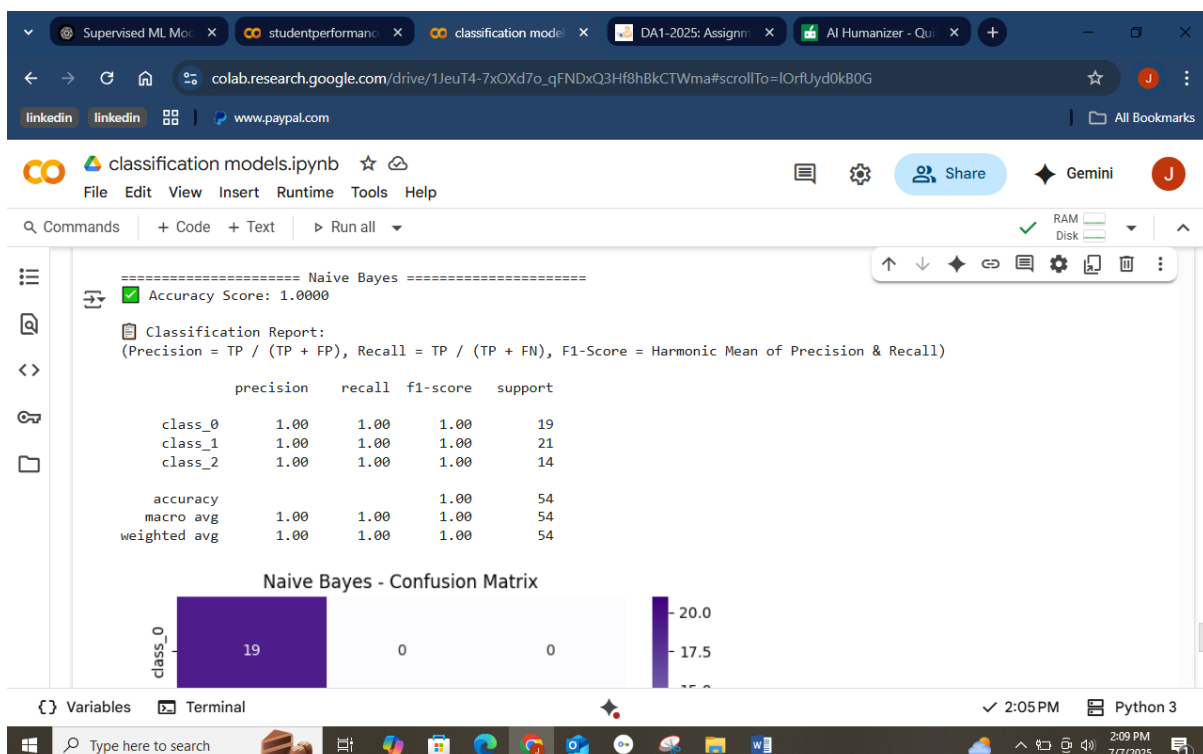


Fig 2.5.5: Output of Naive Bayes

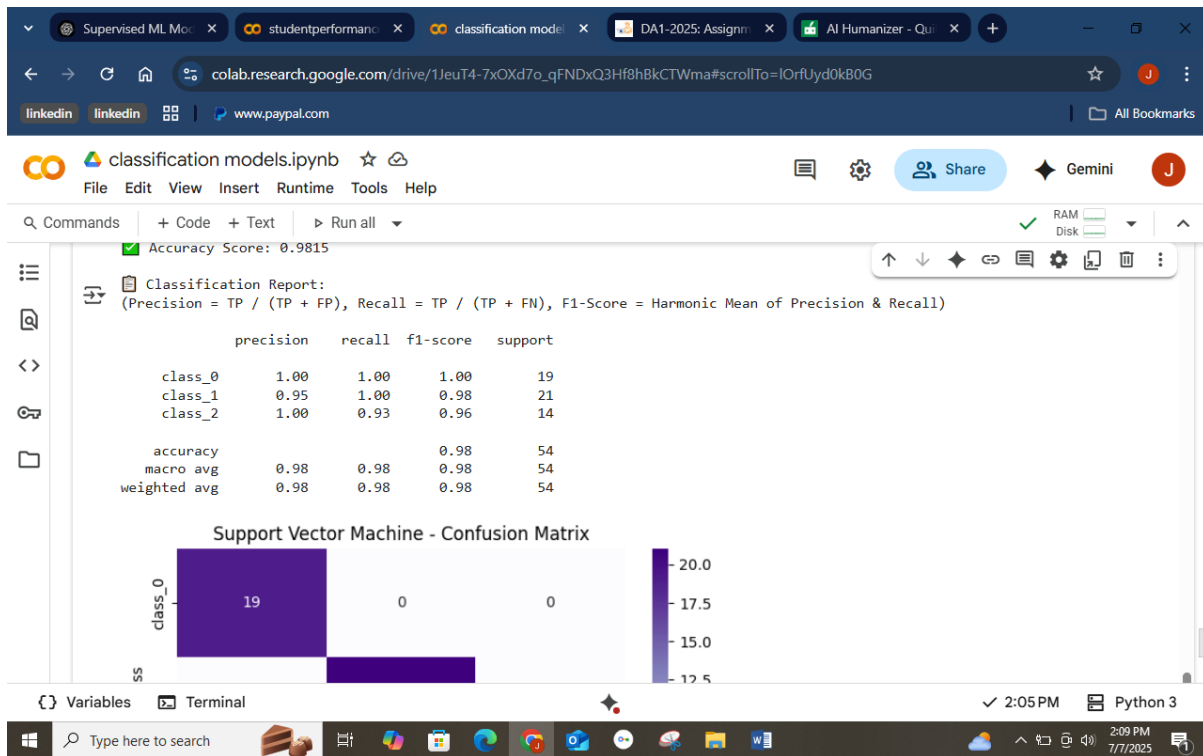


Fig 2.5.6: Output of Super Vector Machine

2.6: Compare the Models

After evaluating all six classification models using consistent metrics, I compare their performance to determine which model performs best for the Wine dataset.

I analyze the results based on:

- **Accuracy scores** to see which model correctly predicted the most test samples overall.
- **Precision, recall, and F1-score** from the classification reports to assess how well each model performed across the individual wine classes.
- **Confusion matrices** to identify where misclassifications occurred and how often the models confused one class for another.

According to **accuracy scores** **Naives Bayes** and **Random Forest** appear to be the best model since they have an accuracy of 1.

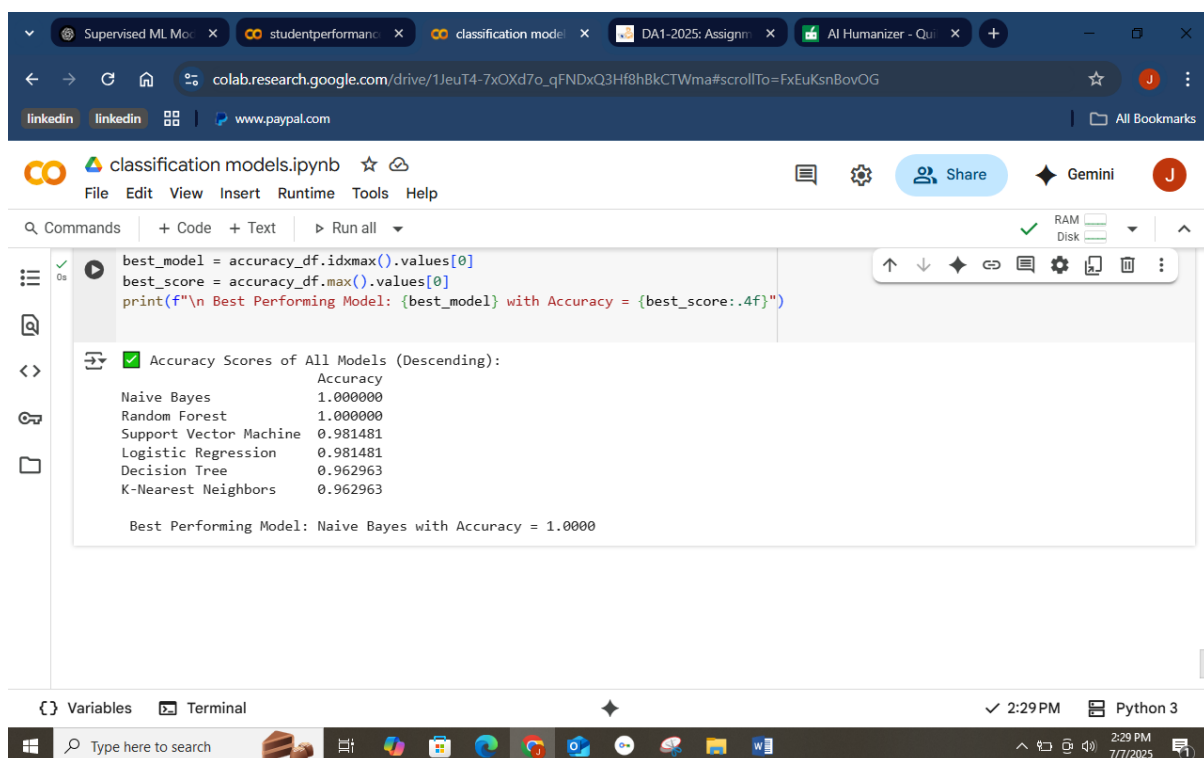


Fig 2.6.1: Output of accuracy scores of all models

According to **Precision, recall, and F1-score**

❖ Random Forest

Perfect scores (1.000) in all metrics across all classes.

Conclusion: Outstanding performance, overfitting is possible but not likely here due to the consistent high recall, precision, and F1.

❖ Naive Bayes

Also shows perfect scores (1.000) across the board.

This is unexpectedly high for a simple model that assumes feature independence.

Conclusion: Performs surprisingly well on this dataset, likely due to the clear class separation in Wine data.

❖ Support Vector Machine (SVM)

Very high scores overall.

Slight drop in recall for class_2 (0.929), suggesting a few false negatives.

Conclusion: Very strong model, slightly underperforms in class_2 recall compared to Random Forest.

❖ Logistic Regression

High precision for all classes.

Slight drop in recall for class_1 (0.952), which affects F1-score.

Conclusion: Performs well, but not as robust as RF, NB, or SVM.

❖ K-Nearest Neighbors (KNN)

Consistent performance across all metrics.

Slightly lower precision and recall for class_1 and class_2.

Conclusion: Good but more sensitive to class boundaries and feature scaling.

❖ Decision Tree

Solid scores, especially in class_1 and class_2.

Lower precision and recall in class_0 (0.947), which lowers its F1.

Conclusion: Performs well but not as balanced as RF or SVM.

Model	Precision (Avg)	Recall (Avg)	F1-Score (Avg)
Random Forest	1.000	1.000	1.000
Naive Bayes	1.000	1.000	1.000
Support Vector Machine	~0.985	~0.976	~0.976
Logistic Regression	~0.978	~0.984	~0.981
K-Nearest Neighbors	~0.961	~0.968	~0.963
Decision Tree	~0.967	~0.992	~0.962

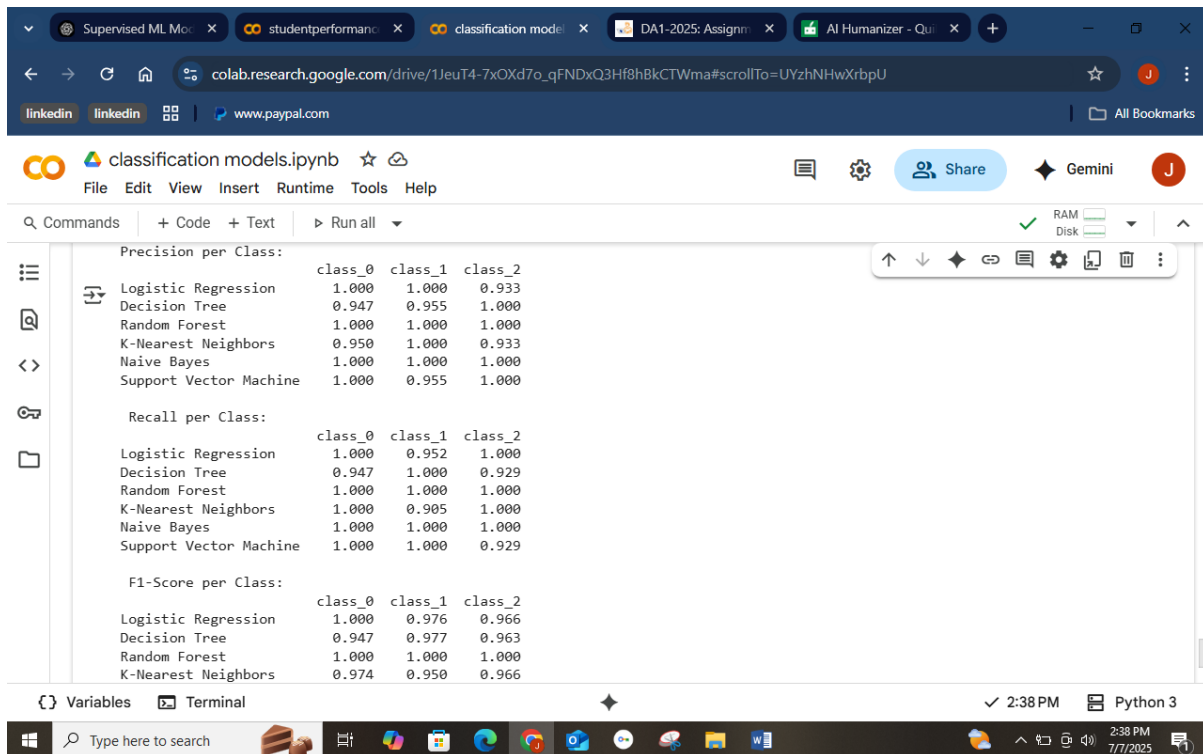


Fig 2.6.2: Output of Precision, recall, and F1-score

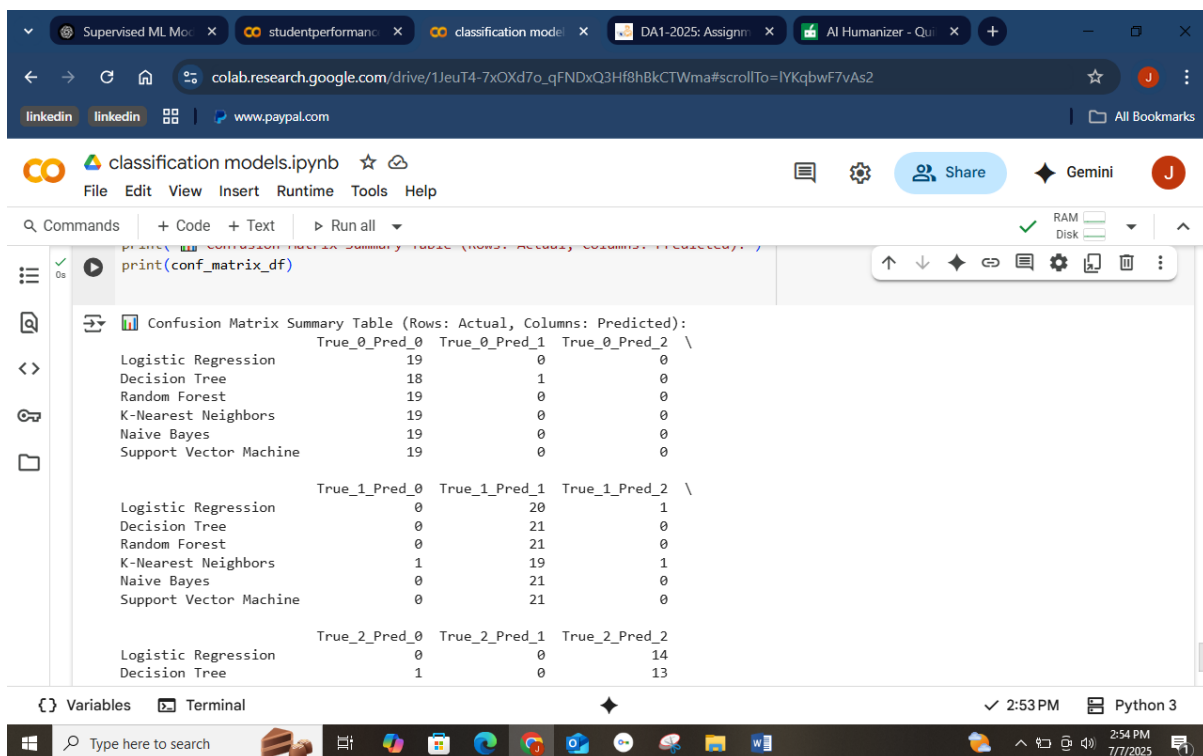


Fig 2.6.3: Output of Confusion matrices

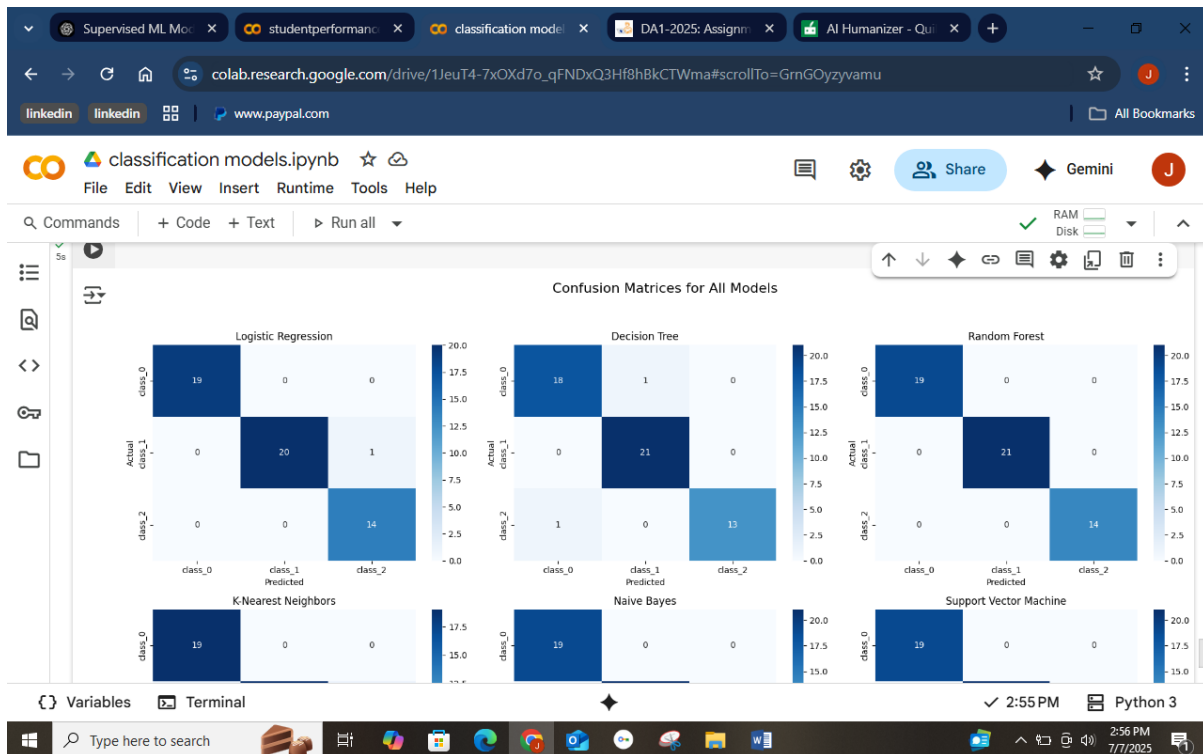


Fig 2.6.4: Output of Confusion matrices for all models using heatmap

- **Diagonal cells** (top-left to bottom-right): Correct predictions.
- **Off-diagonal cells**: Misclassifications — look here to see which classes the model confuses.

❖ Logistic Regression

Very strong performance

Minor confusion: 1 sample from class_1 misclassified as class_2

❖ Decision Tree

Solid performance

A bit more misclassification than Logistic Regression

Some confusion between class_0 and class_1, class_2

❖ Random Forest

Perfect classification

No misclassification this is ideal

Clearly the best performing model based on the confusion matrix

❖ **K-Nearest Neighbors (KNN)**

Very strong, but class_1 is less stable (2 misclassified)

Good performance but slightly worse than Logistic Regression and RF

❖ **Naive Bayes**

Perfect classification

Matches Random Forest's performance

Surprising for Naive Bayes, which typically assumes independence between features

❖ **Support Vector Machine (SVM)**

Near-perfect

Minor issue: 1 sample from class_2 misclassified as class_1

Final Conclusion

Random Forest is the best model overall because it delivers perfect classification with no trade-offs — combining excellent accuracy, strong generalization, and robustness to complex relationships in the data.

Model	Notes
Naive Bayes	Also perfect scores — but simplistic assumptions (e.g., feature independence) mean it's less reliable for complex datasets.
SVM	Very strong, with only one misclassification , but requires feature scaling , and tuning can be complex.
Logistic Regression	Solid baseline, slight drop in class_1 recall and class_2 precisions. Easier to implement but not top performer.
K-Nearest Neighbors	Performs decently, but more sensitive to feature scaling and class overlap.
Decision Tree	Good, but more prone to overfitting and has slightly more misclassifications.

2.7: Share Your Work

To showcase the skills and knowledge applied in this project, the complete code, analysis, and results have been documented and shared publicly. Sharing the project not only demonstrates proficiency in supervised machine learning but also helps build a professional portfolio for future opportunities.

Link to Code:

https://colab.research.google.com/drive/1JeuT4-7xOXd7o_qFNDxQ3Hf8hBkCTWma?usp=sharing

Conclusion

In this project, I successfully applied and evaluated six supervised machine learning classification models on the Wine dataset: Logistic Regression, Decision Tree, Random Forest, k-Nearest Neighbors (KNN), Naive Bayes, and Support Vector Machine (SVM). Through data exploration, preprocessing, training, and performance evaluation I gained valuable insights into how different models behave on the same dataset.

By comparing accuracy scores, classification reports, and confusion matrices, I was able to identify the strengths and limitations of each model. While some models performed better in terms of overall accuracy, others hold stronger results in precision or recall for specific classes. These variations highlighted the importance of using multiple metrics when assessing model performance.

Overall, this assignment enhanced my understanding of the classification process from data preparation to model comparison and reinforced the importance of selecting the right algorithm based on the nature of the data and the specific goals of the task.